


**Asignatura**: Programación 2

# Trabajo Práctico FastFood

| **Comisión**: 2603 | **Ciclo lectivo**: 2026 |
|---|---|

**Docentes**: Miguel Silva

Alumnos: Pablo Gustavo García, Vicente Reynaldo Ciancio,
Carla Sofía Corvalán Portillo

**Fecha de entrega**: 2 de Noviembre de 2026

# Índice

2. Repositorio de GitHub....2
2.1 Diagramas UML....2
3. Descripción general del proyecto....3
3.1 Tema asignado....3
3.2 Objetivos del sistema....3
3.3 Actores y roles....3
4. Arquitectura y desarrollo....5
4.1 Arquitectura general....5
4.2 Capas del sistema....5
4.3 Estructura del proyecto....6
4.4 Tecnologías y frameworks....7
4.5 Clases principales y sus responsabilidades....7
4.6 Aplicación de POO y principios SOLID....8
4.7 Ejemplos de código relevantes....9
4.8 Decisiones de implementación....12
5. Casos de uso y diagramas....13
5.1 Diagrama de casos de uso....13
5.2 Descripción de los casos de uso....13
6. Base de datos....15
6.1 Diagrama entidad-relación....15
6.2 Descripción de las tablas....15
6.3 Relaciones....16
6.4 Ejemplo de datos cargados....17
7. Manual de usuario....18
7.1 Cómo acceder al sistema....18
7.2 Funciones comunes a todos los usuarios....18
7.3 Administrador del sistema (SYS_ADMIN)....19
7.4 Gerente (GERENTE)....20
7.5 Vendedor (VENDEDOR)....21
7.6 Cocinero (COCINERO)....22

# 2. Repositorio de GitHub

El proyecto completo y ejecutable (código del backend y del frontend, script de la base de datos, diagramas e instrucciones de ejecución) está publicado en:

https://github.com/CorvalanPortilloINSPT/java-solid-poo-project

El repositorio está organizado así:

| Carpeta | Contenido |
|---|---|
| `backend-fastfood` | API REST (Spring Boot). Reglas de negocio, seguridad y acceso a MySQL. Puerto 8080. |
| `frontend-fastfood` | Aplicación web (Spring Boot + JSP). Pantallas para cada rol; consume la API. Puerto 8081. |
| `database` | `fastfood.sql`: crea la base fastfood (estructura y datos de ejemplo). |
| `docs/uml` | Diagramas UML del sistema (ver apartado 2.1). |
| `ejecutables` | Scripts `.bat` para iniciar el backend y el frontend. |
| `primera-entrega` | Archivos de la primera entrega (aplicación de consola). |
| `README.md` | Requisitos, pasos para importar la base y ejecutar ambos proyectos, usuarios de prueba. |

Los archivos ejecutables (`backend-fastfood-0.0.1-SNAPSHOT.jar` y `frontend-fastfood-0.0.1-SNAPSHOT.war`) se publican en la sección **Releases** del repositorio, versión v1.0: son archivos compilados a partir del código fuente.

En la primera entrega el sistema era una aplicación de consola con persistencia en archivos de texto; sus archivos se conservan en la carpeta `primera-entrega`. Esta entrega migra ese sistema a una arquitectura web, conservando las reglas de negocio del sistema anterior.

## 2.1 Diagramas UML

Los diagramas de clases muestran todas las clases del sistema con sus atributos, métodos y relaciones. Por su tamaño no se pueden leer en una hoja A4, por eso no se incluyen en este informe y se publican en el repositorio, en la carpeta docs/uml. Los archivos `.puml` son el código fuente de cada diagrama en PlantUML y el `.drawio` se edita en app.diagrams.net:

| Diagrama | Archivos en el repositorio |
|---|---|
| Clases del sistema completo (backend y frontend) | FastFood_UML.pdf · FastFood_UML.puml |
| Clases del backend | FastFood_UML_backend.pdf · FastFood_UML_backend.puml |
| Clases del backend (editable en draw.io) | FastFood_backend_UML.drawio |
| Clases del frontend | FastFood_UML_frontend.pdf · FastFood_UML_frontend.puml |
| Arquitectura (Figura 1) | arquitectura.png |
| Casos de uso (Figura 2) | casos_de_uso.png |
| Entidad-relación (Figura 3) | entidad_relacion.png |

# 3. Descripción general del proyecto

## 3.1 Tema asignado

**Tema 13: Gestión Fast Food.** Enunciado entregado por el docente:

> *«Desarrolle un sistema para gestionar la información de un restaurante de comidas rápidas, relativa a los pedidos y a los empleados que los toman y los despachan, teniendo en cuenta que: Cada pedido está compuesto por un conjunto de ítems. Cada ítem puede, a su vez, estar formado por un conjunto de ingredientes, que pueden ser agregados o quitados. Cada día serán asignados por los gerentes los roles de vendedores y de cocineros. Al sistema podrán acceder tres tipos de usuarios: vendedores (que tomarán los pedidos), cocineros (que despacharán los pedidos) y gerentes (que administrarán todo y obtendrán los listados de ventas).»*

## 3.2 Objetivos del sistema

- Registrar pedidos con uno o más productos, permitiendo agregar o quitar ingredientes por ítem y calculando el total automáticamente.
- Organizar el trabajo de la cocina: los pedidos pendientes se listan en orden y el cocinero los despacha.
- Mantener el catálogo: ingredientes, productos y recetas, con la disponibilidad de cada producto calculada por el sistema.
- Gestionar el personal con permisos por rol y asignar el «rol del día» (vendedor o cocinero).
- Brindar al gerente un reporte de ventas filtrable por fechas y por vendedor.
- Conservar el historial: las bajas son lógicas y los precios se guardan al momento de la venta.
- Proteger el acceso: cada usuario se autentica y solo puede usar las funciones de su rol.
- Poder usarse desde cualquier navegador de la red (PC o celular), con la base de datos, el backend y el frontend en equipos distintos.

## 3.3 Actores y roles

Todos los usuarios del sistema son empleados del local. Cada uno tiene un único rol, que define qué pantallas ve y qué operaciones puede realizar:

| Actor (rol) | Descripción | Responsabilidades |
|---|---|---|
| **SYS_ADMIN** | Administrador del sistema. No opera el negocio. | Da de alta, modifica y da de baja a gerentes y a otros administradores. |
| **GERENTE** | Responsable del local. | Gestiona vendedores, cocineros, ingredientes, productos y recetas; asigna el rol del día; consulta y elimina pedidos pendientes; consulta el reporte de ventas. |

| Actor (rol) | Descripción | Responsabilidades |
|---|---|---|
| **VENDEDOR** | Atiende al cliente en el mostrador. | Registra pedidos (con modificaciones de ingredientes) y consulta los pedidos y su detalle. |
| **COCINERO** | Prepara los pedidos. | Consulta los pedidos pendientes con el detalle de lo que hay que preparar y los despacha. |

Vendedor y cocinero son roles intercambiables: el gerente puede pasar a un empleado de un rol al otro según la necesidad del día (consigna del trabajo práctico). Cualquier empleado puede cambiar su propia contraseña.

# 4. Arquitectura y desarrollo

## 4.1 Arquitectura general

El sistema está formado por **dos aplicaciones Spring Boot independientes** y una base de datos MySQL. El backend expone una **API REST** que concentra las reglas de negocio y el acceso a datos; el frontend es una aplicación web que genera las pantallas (HTML con JSP) y obtiene los datos llamando a esa API. Así, la lógica del negocio está en un solo lugar y la API puede ser usada por otros clientes (por ejemplo, Postman o una futura aplicación móvil).

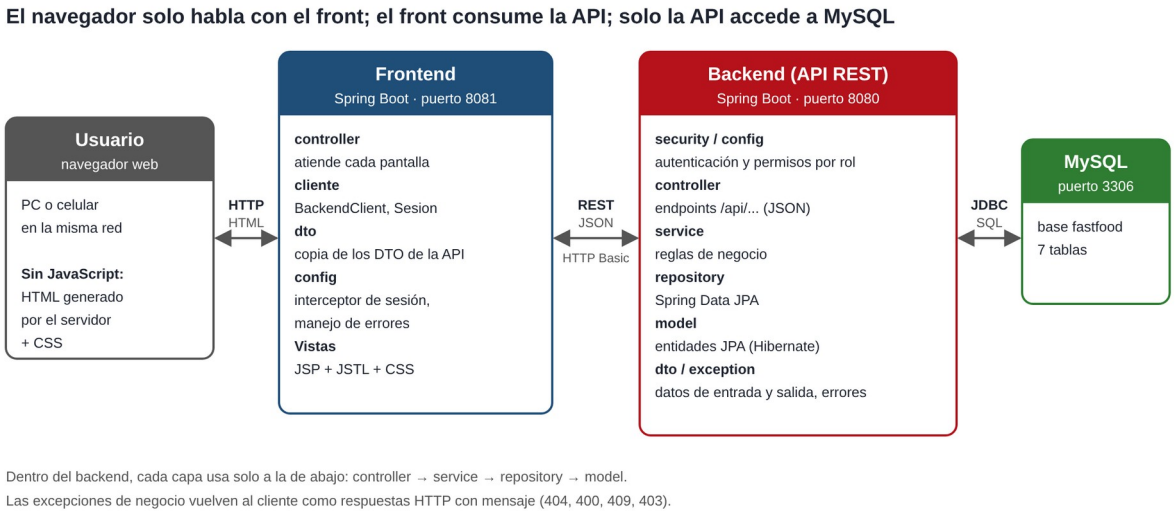



*Figura 1. Arquitectura del sistema: navegador, frontend, backend y base de datos.*

Las tres partes pueden ejecutarse en una misma computadora o en equipos distintos de la misma red: la dirección de la base y la del backend se configuran en `application.properties` o al iniciar cada aplicación.

## 4.2 Capas del sistema

### Backend (API REST)

| Capa (package) | Responsabilidad | No hace |
|---|---|---|
| `controller` | Recibe las peticiones HTTP, convierte el JSON en DTO, delega en el service y devuelve la respuesta con su código de estado. | No contiene lógica de negocio ni accede a la base. |
| `service` | Aplica las reglas de negocio (validaciones, permisos sobre registros, cálculos) y coordina los repositorios. | No conoce HTTP ni JSON. |
| `repository` | Acceso a datos (el DAO de la primera entrega). Interfaces de Spring Data JPA. | No valida reglas. |
| `model` | Entidades JPA que representan el dominio y se mapean a las tablas. | No consulta la base ni conoce la web. |
| `dto` | Objetos de entrada y salida de la API; nunca exponen datos sensibles como la contraseña. | No tienen lógica. |

| Capa (package) | Responsabilidad | No hace |
|---|---|---|
| `exception` | Excepciones de negocio y su conversión centralizada a respuestas HTTP con mensaje. | — |
| `security` | Autenticación: verifica usuario, contraseña y estado del empleado. | — |
| `config` | Autorización por URL y rol, codificación de contraseñas y datos iniciales. | — |

### Frontend (aplicación web)

| Capa | Responsabilidad |
|---|---|
| `controller` | Atiende cada pantalla: recibe los formularios, pide los datos al backend, los coloca en el modelo y elige la vista. |
| `cliente` | Comunicación con la API (`BackendClient`) y manejo de la sesión del usuario (`Sesion`, `Credenciales`). |
| `dto` | Espejo de los DTO del backend, para recibir y enviar JSON. |
| `config` | Comportamiento común a todas las pantallas: control de sesión, manejo de errores y configuración del cliente HTTP. |
| Vistas (JSP + CSS) | Presentación: páginas JSP con JSTL y una hoja de estilos con diseño adaptable a celulares, sin JavaScript. |

## 4.3 Estructura del proyecto

**Backend**

```
backend-fastfood/
└── src/main/java/com/mycompany/backend_fastfood/
    ├── BackendFastfoodApplication.java   (punto de entrada)
    ├── config/        SecurityConfig, InicializadorDatos
    ├── controller/    Auth, Empleado, Ingrediente, Producto, Pedido, Prueba
    ├── dto/           15 clases de entrada y salida (EmpleadoDTO, PedidoRequestDTO, ...)
    ├── exception/     NegocioException y sus 4 subclases, GlobalExceptionHandler
    ├── model/         Persona, Empleado, Ingrediente, Producto, ProductoIngrediente,
    │                  ProductoIngredienteId, Pedido, Item, ItemIngrediente,
    │                  Rol, EstadoPedido, TipoModificacion
    ├── repository/    7 interfaces JpaRepository
    ├── security/      EmpleadoUserDetailsService, TextoPlanoLegacyEncoder
    └── service/       Empleado, Ingrediente, Producto, Pedido, GeneradorId
    src/main/resources/application.properties
```

**Frontend**

```
frontend-fastfood/
├── src/main/java/com/mycompany/frontend_fastfood/
│   ├── FrontendFastfoodApplication.java
│   ├── cliente/       BackendClient, BackendException, Credenciales, Sesion
│   ├── config/        SesionInterceptor, WebConfig, ErroresAdvice, RestTemplateConfig
│   ├── controller/    Auth, Inicio, Empleado, Ingrediente, Producto, Pedido, Prueba
│   └── dto/           copia de los DTO de la API
├── src/main/resources/
│   ├── application.properties          (puerto 8081 y dirección del backend)
│   └── static/css/estilos.css
└── src/main/webapp/WEB-INF/jsp/        17 páginas JSP + fragmentos comunes
```

Los dos proyectos se compilan para **Java 25**. El backend se empaqueta como `.jar` ejecutable y el frontend como `.war` ejecutable, porque las páginas JSP no funcionan dentro de un `.jar`; ambos se inician con `java -jar` y traen Tomcat embebido.

## 4.4 Tecnologías y frameworks

| Tecnología | Uso en el proyecto |
|---|---|
| **Java 25** | Lenguaje de ambos proyectos (desarrollado con JDK 25) |
| **Spring Boot 4.0.8** | Base de las dos aplicaciones: inversión de control, inyección de dependencias y Tomcat embebido. |
| **Spring Web MVC** | API REST (`@RestController`) en el backend; controladores de pantallas (`@Controller`) en el frontend. |
| **Spring Data JPA + Hibernate** | Mapeo objeto-relacional de las entidades y repositorios sin SQL manual. |
| **Spring Security** | Autenticación HTTP Basic contra la tabla de empleados, permisos por rol y contraseñas con BCrypt. |
| **MySQL 8** + Connector/J | Base de datos relacional `fastfood`. |
| **Jackson** | Conversión automática entre objetos Java y JSON (incluido en Spring Web). |
| **RestTemplate** | Cliente HTTP del frontend para consumir la API. |
| **JSP + JSTL** | Generación de las páginas en el servidor. |
| **CSS** | Estilos y diseño adaptable a celulares (menú desplegable sin JavaScript). |
| **Lombok** | Genera getters, setters y constructores de los DTO. |
| **SLF4J** | Registro de eventos (logs) de altas, bajas, errores y accesos. |
| **Maven** | Gestión de dependencias y compilación (Maven Wrapper incluido). |
| Herramientas | NetBeans y Visual Studio Code, MySQL Workbench / Laragon, Postman, Git y GitHub. |

## 4.5 Clases principales y sus responsabilidades

Las tablas siguientes resumen las clases principales. Los diagramas de clases completos del backend y del frontend están en el repositorio (apartado 2.1).

### Backend

| Clase | Responsabilidad |
|---|---|
| `Persona` (abstracta) / `Empleado` | Datos personales, usuario, contraseña y rol; `Empleado` agrega el estado activo. Se guardan en una sola tabla (herencia SINGLE_TABLE). |
| `Ingrediente`, `Producto` | Catálogo. `Producto` distingue `activo` (existe en el catálogo) de `disponible` (se puede vender; lo calcula el sistema). |
| `ProductoIngrediente` + `ProductoIngredienteId` | Receta: qué ingredientes lleva cada producto y cuáles son obligatorios. Clave compuesta (producto, ingrediente). |
| `Pedido`, `Item`, `ItemIngrediente` | Pedido con sus ítems y las modificaciones de cada ítem (composición). Copian los precios al momento de la venta y calculan subtotales y total. |

| Clase | Responsabilidad |
|---|---|
| `EmpleadoService` | Alta, modificación y baja de empleados según quién opera; rol del día; cambio de contraseña; usuario administrador inicial. |
| `IngredienteService`, `ProductoService` | ABM del catálogo y de las recetas; recálculo de la disponibilidad; bajas solo si no afectan pedidos pendientes ni recetas activas. |
| `PedidoService` | Registro de pedidos con todas sus validaciones, despacho, eliminación de pendientes y datos del reporte de ventas. |
| `GeneradorId` | Genera los identificadores con el formato del sistema anterior (E0001, ING01, PR001, PE001...). |
| `SecurityConfig` | Define qué rol puede usar cada endpoint y cómo se verifican las contraseñas. |
| `EmpleadoUserDetailsService` | Busca al empleado al iniciar sesión, bloquea a los dados de baja y traduce el rol en permisos. |
| `GlobalExceptionHandler` | Convierte cada excepción de negocio en una respuesta HTTP con mensaje (404, 400, 409, 403). |

### Frontend

| Clase | Responsabilidad |
|---|---|
| `BackendClient` | Único punto de contacto con la API: envía las credenciales, convierte el JSON y traduce los errores a mensajes para el usuario. |
| `Sesion` | Guarda en la sesión HTTP al usuario logueado, sus credenciales y el pedido que el vendedor está armando. |
| `SesionInterceptor` | Antes de cada pantalla verifica que haya sesión; si no, redirige al login. |
| `ErroresAdvice` | Manejo común de errores: sesión vencida → login; error inesperado → página de error. |
| Controladores (`PedidoController`, etc.) | Una clase por área; arman cada pantalla y aplican el patrón Post/Redirect/Get con mensajes de éxito o error. |

## 4.6 Aplicación de POO y principios SOLID

Los principios descritos en la primera entrega se mantienen y ahora se apoyan en el framework:

| Principio | Aplicación concreta |
|---|---|
| **Abstracción** | `Persona` es abstracta: solo existen empleados. `NegocioException` es abstracta: solo se lanzan sus subclases. |
| **Encapsulamiento** | Atributos privados; solo hay setters para lo que puede cambiar (no existe `setDNI` ni `setID`). `Pedido.getItems()` devuelve una lista de solo lectura. |
| **Herencia** | `Empleado extends Persona`; cuatro excepciones heredan de `NegocioException`. |
| **Polimorfismo** | `GlobalExceptionHandler` responde distinto según la excepción; Spring Security usa un `PasswordEncoder` que puede ser BCrypt o el de texto plano heredado. |

| Principio | Aplicación concreta |
|---|---|
| **Responsabilidad única (S)** | Cada capa tiene una sola función: controller (HTTP), service (reglas), repository (datos). |
| **Abierto/cerrado (O)** | Se agregan pantallas o clientes de la API sin modificar las reglas de negocio. |
| **Sustitución de Liskov (L)** | Un `Empleado` se usa en cualquier lugar donde se espera una `Persona`. |
| **Segregación de interfaces (I)** | Un repositorio por entidad, con solo las consultas que esa entidad necesita. |
| **Inversión de dependencias (D)** | Los services dependen de interfaces de repositorio; Spring inyecta las implementaciones por constructor. |

## 4.7 Ejemplos de código relevantes

**Entidad con composición y reglas propias.** El pedido guarda sus ítems y solo admite el cambio de estado PENDIENTE → DESPACHADO:

**model/Pedido.java (fragmento)**

```
@Entity
@Table(name = "pedido")
public class Pedido {
    @Id
    private String ID;
    private LocalDate fecha;
    @Enumerated(EnumType.STRING)
    private EstadoPedido estado;
    @ManyToOne
    @JoinColumn(name = "empleadoVendedor_ID")
    private Empleado empleadoVendedor;
    @ManyToOne
    @JoinColumn(name = "empleadoCocinero_ID")
    private Empleado empleadoCocinero;
    @OneToMany(mappedBy = "pedido", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<Item> items = new ArrayList<>();
    ...
    public void despachar(Empleado cocinero) {
        if (!isPendiente()) {
            throw new IllegalStateException(
                    "Solo se puede despachar un pedido PENDIENTE");
        }
        this.estado = EstadoPedido.DESPACHADO;
        this.empleadoCocinero = cocinero;
    }

    public double calcularTotal() {
        return items.stream().mapToDouble(Item::calcularSubtotal).sum();
    }
}
```

**Regla de negocio en el service.** Un ingrediente no puede darse de baja si forma parte de una receta activa o de un pedido pendiente; la baja es lógica:

**service/IngredienteService.java (fragmento)**

```
@Transactional
public void darDeBaja(String id) {
    Ingrediente ingrediente = buscar(id);
    if (!ingrediente.isActivo()) {
        throw new ReglaNegocioException(
                "El ingrediente ya está dado de baja");
    }
    if (productoIngredienteRepository
            .existsByIngrediente_IDAndProducto_ActivoTrue(id)) {
        throw new ReglaNegocioException(
                "No se puede dar de baja el ingrediente porque forma parte "
                + "de la receta de uno o más productos activos. ...");
    }
    if (itemIngredienteRepository.existsByIngrediente_IDAndItem_Pedido_Estado(
            id, EstadoPedido.PENDIENTE)) {
        throw new ReglaNegocioException(
                "No se puede dar de baja el ingrediente porque está "
                + "implicado en pedidos pendientes.");
    }
    ingrediente.setActivo(false);
    ingredienteRepository.save(ingrediente);
}
```

**Repositorio con Spring Data JPA.** Las consultas simples se derivan del nombre del método; el reporte usa una consulta JPQL con filtros opcionales:

**repository/PedidoRepository.java (fragmento)**

```
public interface PedidoRepository extends JpaRepository<Pedido, String> {

    List<Pedido> findByEstadoOrderByFechaAscIDAsc(EstadoPedido estado);

    @Query("""
            SELECT p FROM Pedido p
            WHERE p.estado = :estado
              AND (:desde IS NULL OR p.fecha >= :desde)
              AND (:hasta IS NULL OR p.fecha <= :hasta)
              AND (:idVendedor IS NULL OR p.empleadoVendedor.ID = :idVendedor)
            ORDER BY p.fecha ASC, p.ID ASC
            """)
    List<Pedido> buscarParaReporte(@Param("estado") EstadoPedido estado,
            @Param("desde") LocalDate desde, @Param("hasta") LocalDate hasta,
            @Param("idVendedor") String idVendedor);
}
```

**Permisos por rol.** Cada endpoint se habilita solo para los roles que corresponden; lo que no está listado queda bloqueado:

**config/SecurityConfig.java (fragmento)**

```
.requestMatchers(HttpMethod.GET, "/api/pedidos/reporte").hasRole(GERENTE)
.requestMatchers(HttpMethod.GET, "/api/pedidos").hasAnyRole(GERENTE, VENDEDOR)
.requestMatchers(HttpMethod.GET, "/api/pedidos/**")
    .hasAnyRole(GERENTE, VENDEDOR, COCINERO)
.requestMatchers(HttpMethod.POST, "/api/pedidos").hasRole(VENDEDOR)
.requestMatchers(HttpMethod.PUT, "/api/pedidos/*/despacho").hasRole(COCINERO)
.requestMatchers(HttpMethod.DELETE, "/api/pedidos/*").hasRole(GERENTE)
.anyRequest().denyAll()
```

**Manejo centralizado de errores.** Los services solo lanzan la excepción; esta clase decide el código HTTP y el mensaje:

**exception/GlobalExceptionHandler.java (fragmento)**

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RecursoNoEncontradoException.class)
    public ProblemDetail manejarNoEncontrado(RecursoNoEncontradoException ex) {
        return ProblemDetail.forStatusAndDetail(
                HttpStatus.NOT_FOUND, ex.getMessage());
    }

    @ExceptionHandler(ReglaNegocioException.class)
    public ProblemDetail manejarReglaNegocio(ReglaNegocioException ex) {
        return ProblemDetail.forStatusAndDetail(
                HttpStatus.CONFLICT, ex.getMessage());
    }
    ...
}
```

**Consumo de la API desde el frontend.** Un único método genérico agrega las credenciales, llama al backend y traduce cualquier error en un mensaje para la pantalla:

**cliente/BackendClient.java (fragmento)**

```
private <T> T llamar(HttpMethod metodo, String ruta, Object body,
                     Class<T> tipo, Credenciales credenciales) {
    HttpHeaders headers = new HttpHeaders();
    headers.setBasicAuth(credenciales.usuario(), credenciales.password(),
            StandardCharsets.UTF_8);
    ...
    try {
        return restTemplate
                .exchange(backendUrl + ruta, metodo, request, tipo)
                .getBody();
    } catch (HttpStatusCodeException e) {
        throw new BackendException(status, mensajeDeError(e, status));
    } catch (ResourceAccessException e) {
        throw new BackendException(BackendException.SIN_CONEXION,
                "No se pudo conectar con el backend. Verifique que esté iniciado.");
    }
}
```

**Vista JSP con JSTL.** La pantalla del cocinero recorre los pedidos pendientes y muestra el botón de despacho solo a ese rol:

**WEB-INF/jsp/pendientes.jsp (fragmento)**

```
<c:forEach var="pedido" items="${pedidos}">
    <section class="tarjeta">
        <h2>Pedido <c:out value="${pedido.id}"/></h2>
        <%@ include file="/WEB-INF/jsp/fragmentos/detallePedido.jspf" %>
        <c:if test="${puedeDespachar}">
            <form method="post" action="${ctx}/pedidos/${pedido.id}/despachar">
                <button type="submit">
                    Despachar pedido <c:out value="${pedido.id}"/>
                </button>
            </form>
        </c:if>
```

```
    </section>
</c:forEach>
```

## 4.8 Decisiones de implementación

| Decisión | Motivo |
|---|---|
| Backend y frontend separados, comunicados por REST | Separación de responsabilidades: las reglas viven solo en la API y se pueden sumar otros clientes. |
| DTO en lugar de devolver entidades | No exponer datos sensibles, evitar ciclos al convertir a JSON y desacoplar la API de la base. |
| Baja lógica (`activo = false`) | Los pedidos históricos siguen referenciando a empleados, productos e ingredientes dados de baja. |
| Precios copiados al vender | `Item` e `ItemIngrediente` guardan el precio del momento: el historial y el reporte no cambian si cambian los precios. |
| IDs con formato propio (E0001, PR001...) | Compatibilidad con los datos y el formato del sistema anterior. |
| Contraseñas con BCrypt | Las contraseñas nuevas se guardan cifradas; las heredadas en texto plano se siguen aceptando para no perder usuarios. |
| Validaciones en el backend | Aunque el frontend filtre opciones, la API vuelve a validar todo (por ejemplo, que el producto esté disponible). |
| Interfaz sin JavaScript | Pantallas generadas en el servidor; el diseño adaptable y el menú del celular se resuelven solo con CSS. |
| Ingredientes extra: solo suma lo agregado | Igual que en plataformas como PedidosYa o Mercado Libre, al personalizar un producto se cobra cada ingrediente agregado y quitar un ingrediente no descuenta. A diferencia de ellas, el sistema no permite quitar los ingredientes obligatorios de la receta, para no vender un producto vacío. |
| Usuarios iniciales por rol | Al iniciar, si no hay un administrador o un gerente activo, el backend crea uno (`sysadmin` y `admin`), para que nunca quede el sistema sin alguien que pueda dar de alta empleados. |
| Bloqueo de los caracteres < > { } | Se rechazan en el login y en los datos de empleados (en el navegador, en el frontend y en la API) para evitar que se ingrese código; además, todas las vistas escapan el texto que muestran. |
| Páginas sin caché | El frontend indica al navegador que no guarde las páginas. Volver con Atrás a la pantalla de login cierra la sesión, y las pantallas anteriores ya no permiten operar. |

# 5. Casos de uso y diagramas

## 5.1 Diagrama de casos de uso

Los cuatro roles son especializaciones del actor Empleado, que concentra los casos de uso comunes (iniciar sesión, cambiar la contraseña y cerrar sesión).

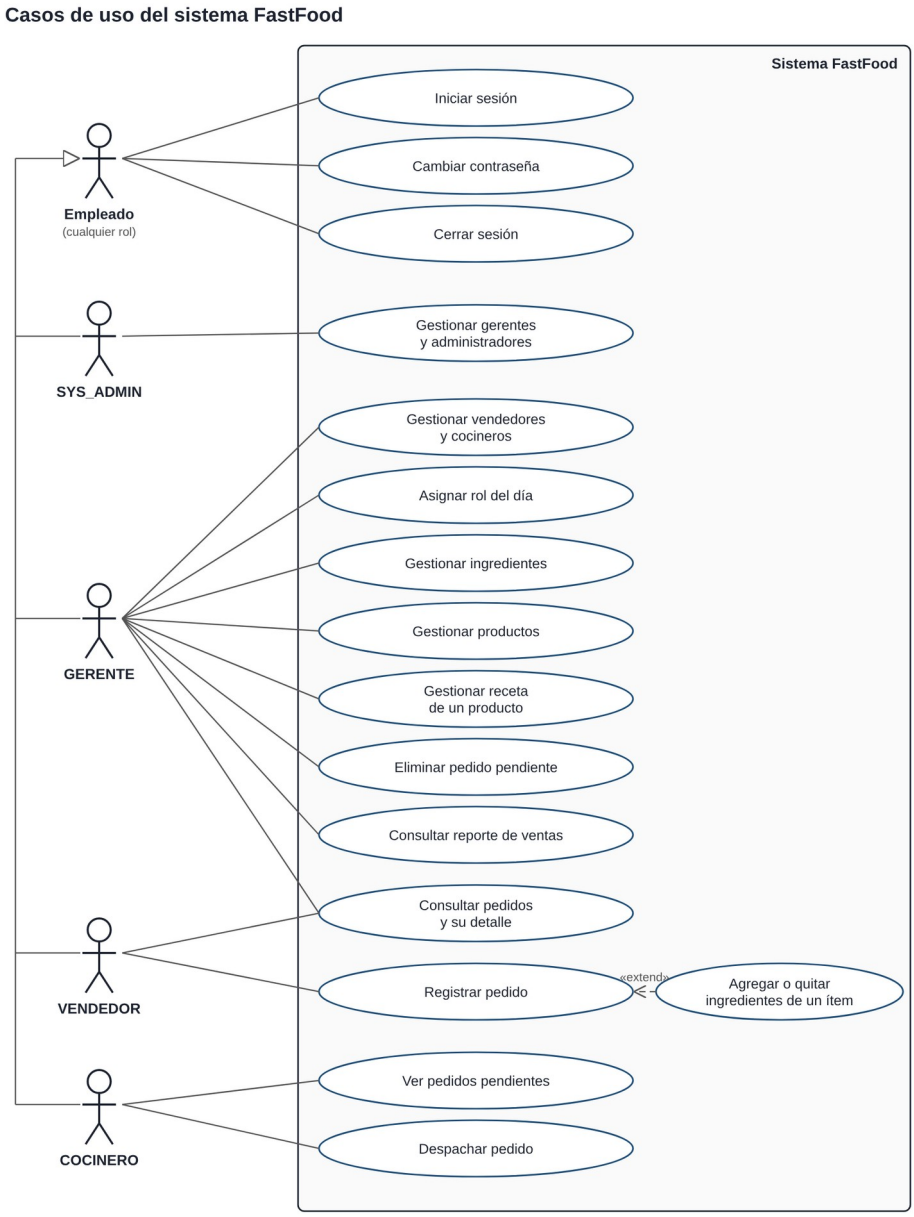



*Figura 2. Diagrama de casos de uso.*

## 5.2 Descripción de los casos de uso

| CU | Caso de uso | Actor | Descripción y reglas principales |
|---|---|---|---|
| 01 | Iniciar sesión | Empleado | Ingresa usuario y contraseña. Un empleado dado de baja no puede ingresar. Al entrar ve el menú de su rol. |

| CU | Caso de uso | Actor | Descripción y reglas principales |
|---|---|---|---|
| 02 | Cambiar contraseña | Empleado | Indica la contraseña actual y la nueva (dos veces). La nueva se guarda cifrada. |
| 03 | Cerrar sesión | Empleado | Finaliza la sesión y vuelve a la pantalla de login. |
| 04 | Gestionar gerentes y administradores | SYS_ADMIN | Alta, modificación y baja de GERENTE y SYS_ADMIN. DNI y usuario únicos; el DNI no se modifica; no puede darse de baja a sí mismo ni al último gerente o administrador activo. |
| 05 | Gestionar vendedores y cocineros | GERENTE | Alta, modificación y baja lógica de VENDEDOR y COCINERO, con las mismas reglas de unicidad. |
| 06 | Asignar rol del día | GERENTE | Pasa a un empleado activo de vendedor a cocinero o viceversa. |
| 07 | Gestionar ingredientes | GERENTE | Alta, modificación y baja. Descripción única; precio no negativo; no se da de baja si está en la receta de un producto activo o en pedidos pendientes. |
| 08 | Gestionar productos | GERENTE | Alta, modificación y baja. Un producto nuevo no está disponible hasta tener receta; no se da de baja si está en pedidos pendientes. |
| 09 | Gestionar receta de un producto | GERENTE | Agrega ingredientes activos (marcando si son obligatorios) o quita los opcionales. Un producto está disponible si tiene al menos un ingrediente obligatorio. |
| 10 | Consultar pedidos y su detalle | GERENTE, VENDEDOR | Lista todos los pedidos con fecha, estado, vendedor, cocinero y total; permite ver el detalle de cada uno. |
| 11 | Eliminar pedido pendiente | GERENTE | Elimina un pedido que todavía no fue despachado, junto con sus ítems. |
| 12 | Consultar reporte de ventas | GERENTE | Muestra los pedidos despachados con cantidad y total vendido, filtrando por rango de fechas y por vendedor. |
| 13 | Registrar pedido | VENDEDOR | Arma el pedido agregando productos disponibles y cantidades; al confirmar, el sistema valida todo, asigna el ID y calcula el total. Queda PENDIENTE. |
| 14 | Agregar o quitar ingredientes de un ítem | VENDEDOR | Extiende CU 13. Agregar suma el precio del ingrediente; no se puede quitar un ingrediente obligatorio ni más cantidad de la que tiene el producto. |
| 15 | Ver pedidos pendientes | COCINERO | Lista los pedidos pendientes, del más antiguo al más nuevo, con productos y modificaciones a preparar. |
| 16 | Despachar pedido | COCINERO | Marca un pedido pendiente como DESPACHADO y registra qué cocinero lo despachó. |

# 6. Base de datos

## 6.1 Diagrama entidad-relación

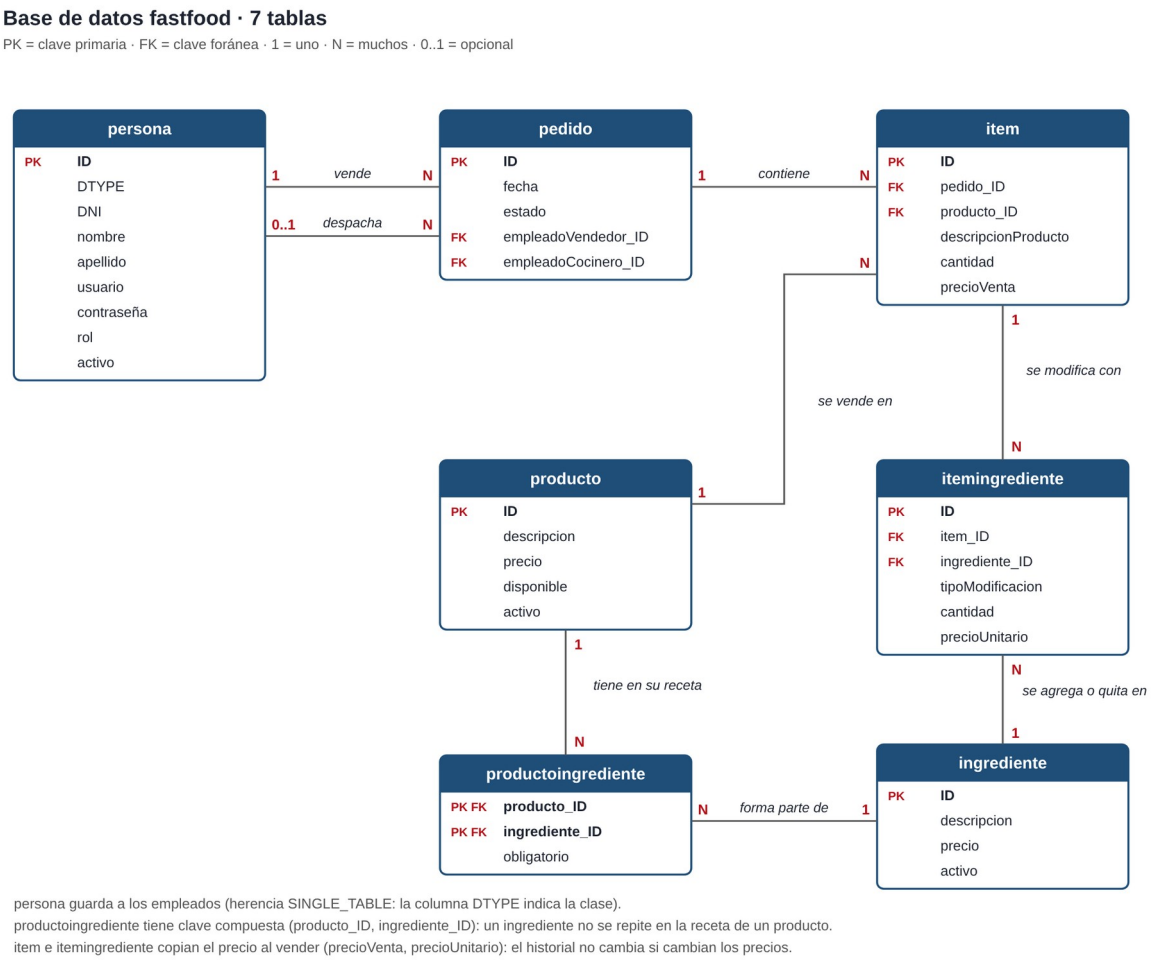



*Figura 3. Diagrama entidad-relación de la base fastfood.*

## 6.2 Descripción de las tablas

La base `fastfood` (MySQL) tiene siete tablas. Hibernate trabaja en modo de validación (`ddl-auto=validate`): no crea ni modifica tablas, solo verifica que coincidan con las entidades. Los nombres de columna son iguales a los atributos de las clases.

| Tabla | Columnas | Descripción |
|---|---|---|
| `persona` | **ID** (PK), DTYPE, DNI, nombre, apellido, usuario, contraseña, rol, activo | Empleados. DTYPE indica la clase (herencia SINGLE_TABLE). rol: VENDEDOR, COCINERO, GERENTE o SYS_ADMIN. activo: baja lógica. |
| `ingrediente` | **ID** (PK), descripcion, precio, activo | Ingredientes; el precio es el costo de agregarlo a un producto. |

| Tabla | Columnas | Descripción |
|---|---|---|
| `producto` | **ID** (PK), descripcion, precio, disponible, activo | Productos a la venta. disponible lo calcula el sistema a partir de la receta. |
| `productoingrediente` | **producto_ID** (PK, FK), **ingrediente_ID** (PK, FK), obligatorio | Receta de cada producto. Clave compuesta: un ingrediente no se repite en una receta. |
| `pedido` | **ID** (PK), fecha, estado, empleadoVendedor_ID (FK), empleadoCocinero_ID (FK) | Pedidos. estado: PENDIENTE o DESPACHADO. El cocinero queda vacío hasta el despacho. |
| `item` | **ID** (PK), pedido_ID (FK), producto_ID (FK), descripcionProducto, cantidad, precioVenta | Líneas del pedido. Copia la descripción y el precio del producto al vender. |
| `itemingrediente` | **ID** (PK), item_ID (FK), ingrediente_ID (FK), tipoModificacion, cantidad, precioUnitario | Modificaciones de un ítem (AGREGADO o QUITADO), por unidad. Copia el precio del ingrediente. |

## 6.3 Relaciones

| Relación | Cardinalidad | Detalle |
|---|---|---|
| persona → pedido (vende) | 1 a N | Cada pedido tiene un vendedor; un vendedor registra muchos pedidos. |
| persona → pedido (despacha) | 0..1 a N | Cada pedido tiene, como máximo, un cocinero (asignado al despachar). |
| pedido → item | 1 a N (composición) | Los ítems se guardan y se eliminan junto con su pedido. |
| producto → item | 1 a N | Un producto aparece en muchos ítems de distintos pedidos. |
| item → itemingrediente | 1 a N (composición) | Las modificaciones se guardan y se eliminan con su ítem. |
| ingrediente → itemingrediente | 1 a N | Un ingrediente puede agregarse o quitarse en muchos ítems. |
| producto ↔ ingrediente (receta) | N a M | Resuelta con la tabla `productoingrediente`, que además guarda si el ingrediente es obligatorio. |

Conceptualmente hay tres relaciones de muchos a muchos: producto–ingrediente (receta), pedido–producto (ítems) e ítem–ingrediente (modificaciones). Ninguna usa `@ManyToMany`, porque todas tienen datos propios; se modelan como entidades intermedias con dos relaciones `@ManyToOne`. La receta usa clave compuesta porque la pareja no se repite; ítems y modificaciones tienen un ID propio porque representan ventas concretas.

## 6.4 Ejemplo de datos cargados

El script `database/fastfood.sql` carga datos de ejemplo: 7 empleados (uno por rol, más un vendedor y un cocinero extra y un vendedor dado de baja), 22 ingredientes, 13 productos con sus recetas y 10 pedidos (8 despachados y 2 pendientes). Algunos ejemplos:

| Tabla | Ejemplo |
|---|---|
| persona | E0003 · Pérez, Ratón · VENDEDOR — E0004 · Azul, El Tiburón · COCINERO — E0007 · López, Julián · VENDEDOR (inactivo) |
| producto | PR003 · Hamburguesa con panceta · $3.600 — PR012 · Wrap de pollo · activo pero no disponible (sin receta) |
| ingrediente | ING03 · Queso cheddar · $300 — ING07 · Panceta · $450 |
| productoingrediente | PR001 Hamburguesa clásica: Pan y Medallón (obligatorios); Lechuga, Tomate y Salsa especial (opcionales) |
| pedido / item | PE002 · 2026-10-19 · vendedor E0005 · cocinero E0006: Hamburguesa con panceta ($3.600) + Gaseosa cola ($1.200) |
| itemingrediente | En PE002 se agrega Queso cheddar a la hamburguesa (AGREGADO, $300): total $5.100. En PE005 se quita la Lechuga (QUITADO): no descuenta. |

# 7. Manual de usuario

## 7.1 Cómo acceder al sistema

Requisitos: Java 25 o superior y MySQL 8, con la base fastfood creada ejecutando `database/fastfood.sql` (por ejemplo, desde MySQL Workbench). Los archivos `.jar` y `.war` se descargan desde la sección Releases del repositorio.

1. Iniciar el backend con `ejecutables/1-iniciar-backend.bat` (o `java -jar backend-fastfood-0.0.1-SNAPSHOT.jar`, o `mvnw spring-boot:run` desde el proyecto). Si la base está en otra computadora, indicar su dirección con `--spring.datasource.url=...`.
2. Iniciar el frontend con `ejecutables/2-iniciar-frontend.bat` (o `java -jar frontend-fastfood-0.0.1-SNAPSHOT.war`). Si el backend está en otra computadora, indicar su dirección con `--backend.url=http://IP:8080`.
3. Abrir en el navegador `http://localhost:8081`. Desde otro equipo o un celular de la misma red: `http://IP-del-frontend:8081`.
4. Ingresar usuario y contraseña y presionar **Ingresar**. Usuarios de la base de ejemplo: `sysadmin` / `sysadmin123` (SYS_ADMIN), `admin` / `admin123` (GERENTE), `rperez` / `vendedor123` (VENDEDOR) y `tazul` / `cocinero123` (COCINERO). Si falta un administrador o un gerente activo, el sistema crea `sysadmin` o `admin` al iniciar; conviene cambiar esas contraseñas al primer ingreso.

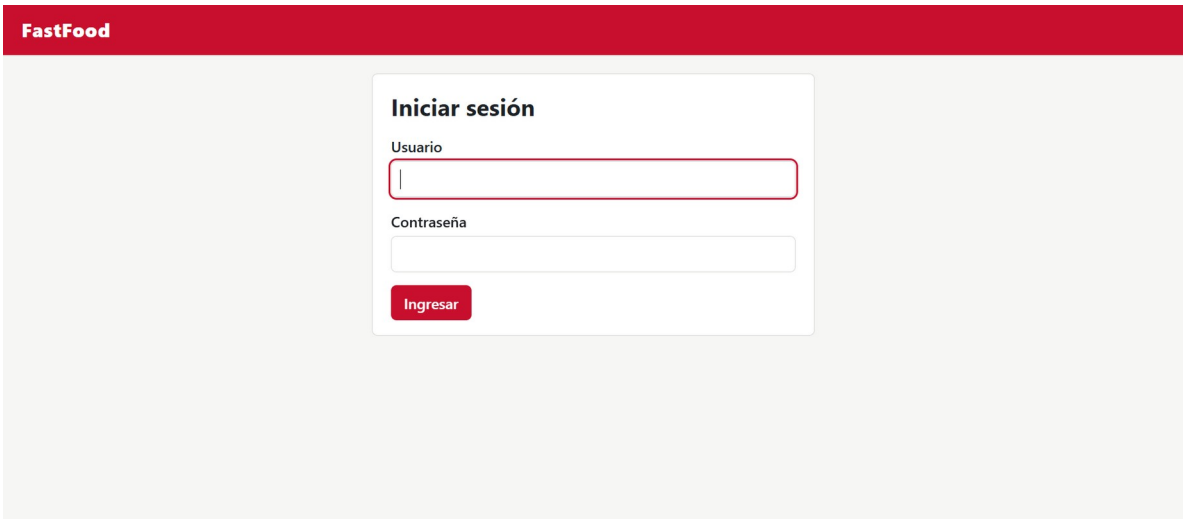



*Figura 4. Pantalla de inicio de sesión.*

## 7.2 Funciones comunes a todos los usuarios

- **Menú de inicio:** muestra solo las opciones del rol del usuario.
- **Encabezado:** nombre del usuario, su rol, el botón **Inicio** y **Cerrar sesión**. En el celular estas opciones se abren con el botón de menú (☰).
- **Cambiar contraseña** (sección «Mi cuenta» del inicio): completar la contraseña actual y la nueva dos veces.

- Después de cada operación se muestra un mensaje de confirmación o el motivo por el que no se pudo realizar.




*Figura 5. Vista en el celular: menú de inicio y menú desplegable del encabezado.*

## 7.3 Administrador del sistema (SYS_ADMIN)

Opción del menú: **Gerentes y administradores**.

1. **Ver el listado** de gerentes y administradores, con su estado.
2. **Nuevo empleado:** completar DNI, nombre, apellido, usuario, contraseña y rol (GERENTE o SYS_ADMIN) y presionar **Guardar**.
3. **Modificar:** cambiar nombre, apellido o usuario. La contraseña es opcional (si se deja vacía, se mantiene). El DNI no se modifica.
4. **Dar de baja:** presionar **Dar de baja** y confirmar. No se puede dar de baja a uno mismo ni al último gerente o administrador activo.

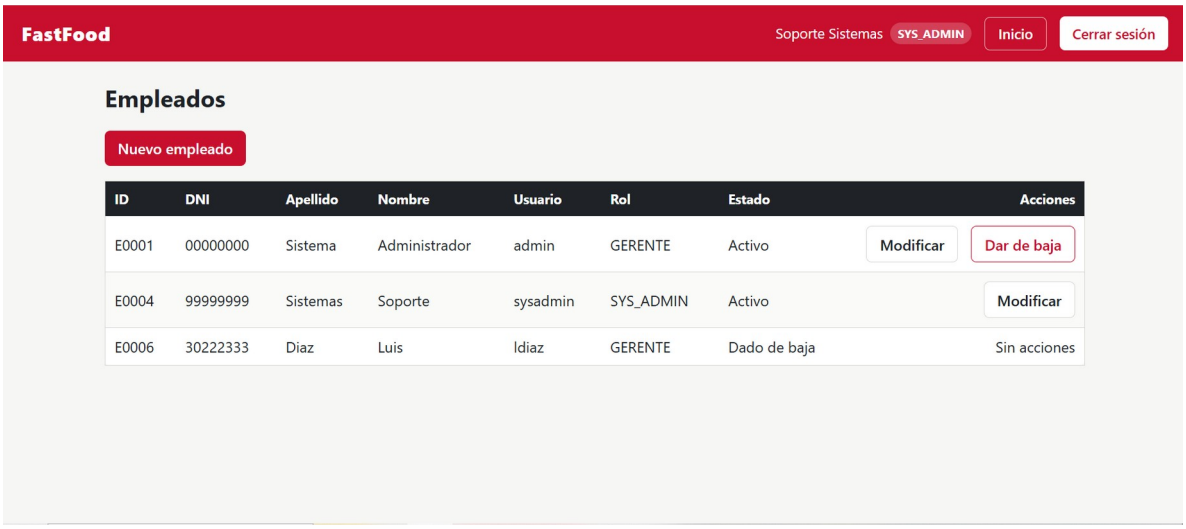



*Figura 6. Listado de gerentes y administradores (usuario SYS_ADMIN).*

## 7.4 Gerente (GERENTE)




*Figura 7. Menú de inicio del gerente.*

### Vendedores y cocineros (rol del día)

- Alta, modificación y baja de vendedores y cocineros, igual que en 7.3.
- **Rol del día:** en el listado, los botones **Pasar a cocinero** / **Pasar a vendedor** cambian el rol del empleado.

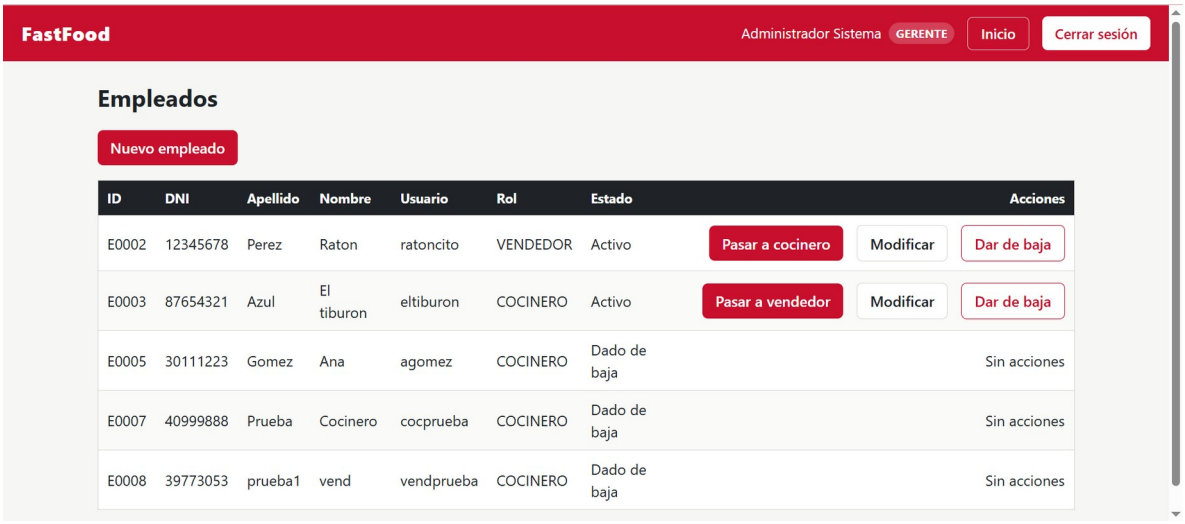



*Figura 8. Listado de vendedores y cocineros con los botones de rol del día.*

### Ingredientes

- **Nuevo ingrediente:** descripción y precio. **Modificar** y **Dar de baja** desde el listado.
- Si el ingrediente está en la receta de un producto activo o en un pedido pendiente, la baja se rechaza con un mensaje.

### Productos y recetas

- **Nuevo producto:** descripción y precio. El producto queda **no disponible** hasta que su receta tenga un ingrediente obligatorio.
- **Receta:** elegir un ingrediente, marcar si es obligatorio y presionar **Agregar**. Los ingredientes opcionales se pueden **Quitar**; los obligatorios no.
- **Modificar** y **Dar de baja** desde el listado (la baja se rechaza si el producto está en pedidos pendientes).




*Figura 9. Receta de un producto.*

### Pedidos

- Listado de todos los pedidos con fecha, estado, vendedor, cocinero y total. **Ver detalle** muestra ítems y modificaciones.
- Los pedidos **pendientes** se pueden eliminar con **Eliminar pedido** (con confirmación).

### Reporte de ventas

- Filtrar por **Desde**, **Hasta** y **Vendedor** (o «Todos») y presionar **Consultar**. **Limpiar filtros** vuelve a mostrar todo.
- Se muestra un resumen (cantidad de pedidos y total vendido) y el listado de pedidos despachados (Figura 10).

**Pedidos**

| Pedido | Fecha | Vendedor | Cocinero | Total |
|---|---|---|---|---|
| PE002 | 2026-10-05 | Perez, Raton (E0002) | Azul, El tiburon (E0003) | $4,500.00 |
| PE003 | 2026-10-05 | Perez, Raton (E0002) | Azul, El tiburon (E0003) | $3,000.00 |
| PE005 | 2026-10-05 | Perez, Raton (E0002) | Azul, El tiburon (E0003) | $1,500.00 |
| PE006 | 2026-10-05 | Perez, Raton (E0002) | Azul, El tiburon (E0003) | $18,000.00 |
| PE007 | 2026-10-08 | Perez, Raton (E0002) | Azul, El tiburon (E0003) | $4,500.00 |
| PE009 | 2026-10-08 | Perez, Raton (E0002) | Azul, El tiburon (E0003) | $6,300.00 |
| PE010 | 2026-10-08 | Perez, Raton (E0002) | Azul, El tiburon (E0003) | $4,800.00 |
| PE011 | 2026-10-08 | Perez, Raton (E0002) | Azul, El tiburon (E0003) | $1,800.00 |
| PE013 | 2026-10-10 | Perez, Raton (E0002) | Azul, El tiburon (E0003) | $2,500.00 |
| PE014 | 2026-10-12 | Perez, Raton (E0002) | Azul, El tiburon (E0003) | $2,500.00 |
| PE015 | 2026-10-12 | Perez, Raton (E0002) | Azul, El tiburon (E0003) | $6,000.00 |

*Figura 10. Reporte de ventas.*

## 7.5 Vendedor (VENDEDOR)

Opciones del menú: **Registrar pedido** y **Consultar pedidos**.

1. En **Agregar producto**, elegir un producto disponible y la cantidad, y presionar **Agregar**. Se pueden sumar varios productos.
2. Para modificar un ítem: elegir la acción (**Agregar** o **Quitar**), el ingrediente y la cantidad por unidad, y presionar **Agregar modificación**. El sistema avisa si se intenta quitar un ingrediente obligatorio o más de lo que tiene el producto.
3. Revisar el **Total estimado**. Con **Quitar ítem** se elimina un producto del pedido; con **Descartar pedido** se empieza de nuevo.
4. Presionar **Confirmar pedido**. El sistema asigna el número de pedido, calcula el total definitivo y lo deja pendiente para la cocina.

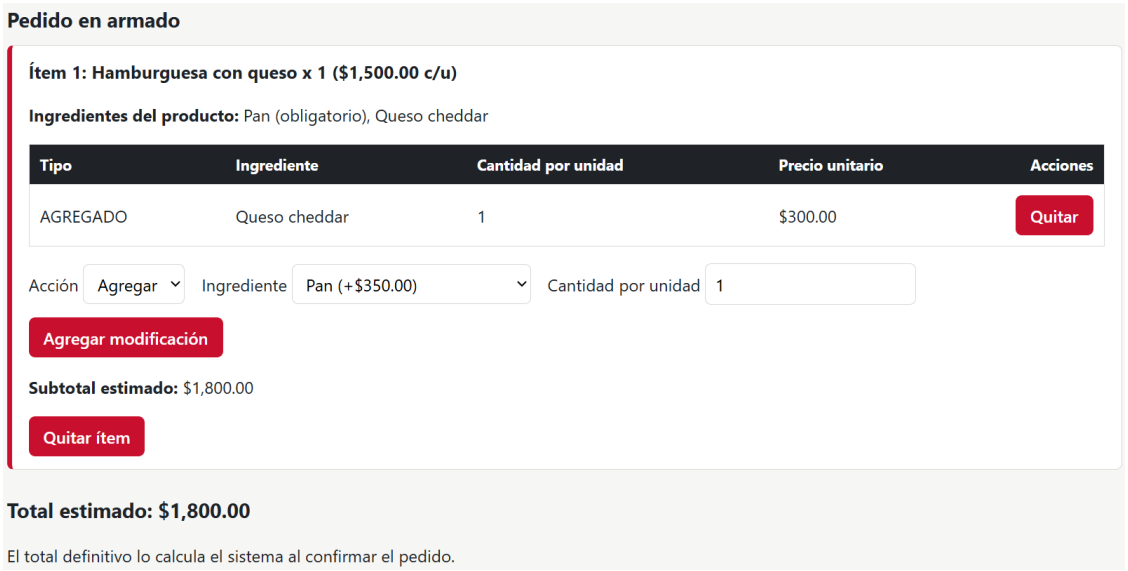



*Figura 11. Pedido en armado con un ingrediente agregado.*

En **Consultar pedidos** el vendedor ve el listado de pedidos y el detalle de cada uno.

## 7.6 Cocinero (COCINERO)

Opción del menú: **Pedidos pendientes**.

- Cada pedido pendiente se muestra con su fecha, vendedor, productos, cantidades y modificaciones (con la cantidad total a preparar).
- Al terminar de prepararlo, presionar **Despachar pedido**. El pedido deja de figurar como pendiente y queda registrado el cocinero que lo despachó.

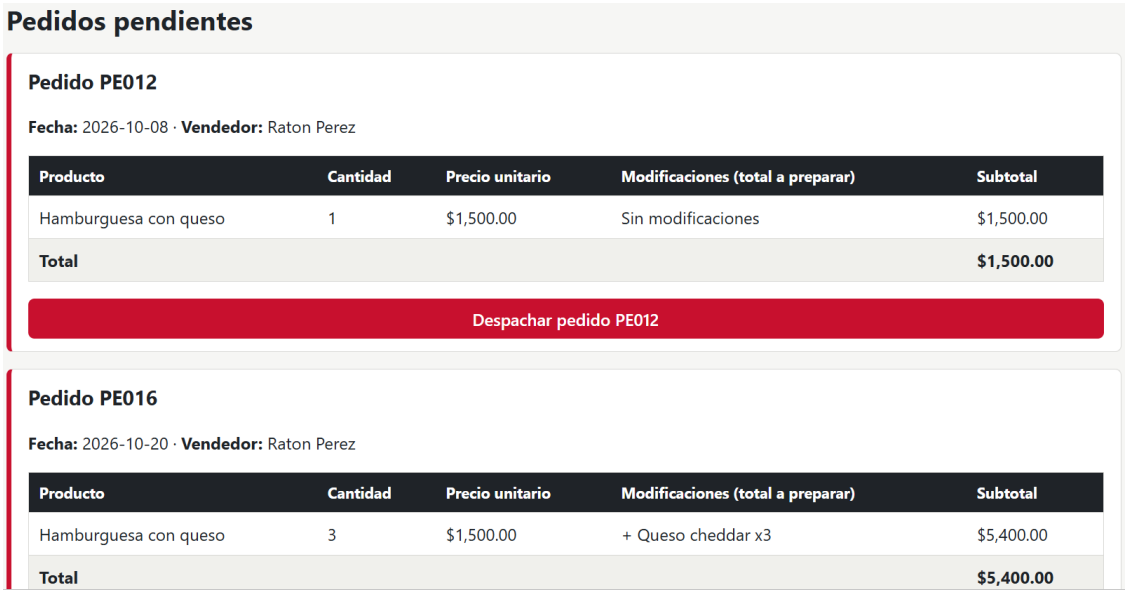



*Figura 12. Pedidos pendientes en la pantalla del cocinero.*